

docs 再構成プラン

decisions : README に画像 / AGENTS.md はパスで記述 / development-rules を skills へ分解 / PR レビュー運用へ

1. 目標の構成

```
health_care_app/  
├─ README.md           ← 人間向け。概要・スクショ・ディレクトリ構成・パスルール表  
├─ AGENTS.md           ← AI 向け入口。Codex が読む。(PROJECT_RULES.md を改名)  
├─ .github/pull_request_template.md  
└─ docs/  
    ├─ README.md       ← 10行程度に瘦せる。docs 配下の索引だけ  
    ├─ product-requirements.md ← そのまま (仕様の正)  
    ├─ development-rules.md ← 分解して削除  
    └─ skills/  
        ├─ clean-architecture/SKILL.md   レイヤー分割と依存の向き  
        ├─ flutter/SKILL.md              Widget・状態管理・コメント方針  
        ├─ app-flow/SKILL.md             画面遷移 (Mermaid)  
        ├─ test/SKILL.md                  どこまで書くか  
        └─ guardrails/SKILL.md            勝手にやっちゃいけない線
```

ファイル	読む人	役割 (1行で言えること)
README.md	人間	何のアプリか。画面はどう見えるか。どこに何を書くか (パスルール表)
AGENTS.md	AI	概要 + 読む順番 + 最上位ルール + 「〇〇する前に △△ を読め」
docs/product-requirements.md	両方	仕様の正 (何を作るか)
docs/skills/<name>/SKILL.md	AI	実装方針の正 (どうするか)。テーマごとに1フォルダ1ファイル

2. development-rules.md をどこへ分解するか

ここが今回の肝です。全部が skill になるわけではありません。「進め方」は skill ではなく入口に置きます。

現在の章	移す先	理由
7. Codexの作業ルール (1タスクずつ / 基本フロー)	AGENTS.md	作業テーマに関係なく毎回効くルールなので入口に置く
8. Humanへ確認する事項	docs/skills/guardrails/SKILL.md	ほぼそのまま使える。加工不要
9. Codexに任せてよい事項	docs/skills/guardrails/SKILL.md	8章の裏返し。同じファイルに置くと判断しやすい
10. Flutter学習のためのルール	docs/skills/flutter/SKILL.md	コードを書くときだけ読めばいい
11. タスク完了時の報告	AGENTS.md	報告の形式は毎回同じなので入口。※ 今この本文は PROJECT_RULES.md L29-38 に散らばっている
12. 次タスクへの移行	AGENTS.md	7章とセット。実質同じルール

移し終わると docs/development-rules.md は空になります。空になったら削除してください。中途半端に残すと「どっちが正か」で必ず事故ります。

分解のときの注意 9章の「Codexに任せてよい事項」には「採用したものをどこに書くか」を必ず足してください。今の書き方（「採用理由を Human へ説明してから進める」）だと口頭で終わって docs に残らず、実際すでに状態管理と写真保存方式の決定がどこにも記録されていません。→ 「採用したものは該当する SKILL.md に理由つきで1行追記する」に書き換える。

3. AGENTS.md のひな形（パスで記述）

AGENTS.md

このプロジェクトで作業する AI エージェント共通のルールです。
各ドキュメントはファイル名ではなくパスで指定しています。記載のパスを直接開いてください。

プロジェクト概要

赤ちゃん報告アプリ（仮）。診察時に医師へ生活状況を説明する手間を減らす健康ログアプリ。
血糖値・測定タイミング・メモ・食事写真を1件の記録として保存し、カレンダーから日付ごとに見返す。
現在は Prototype 1 (MVP)。ログイン・AI 予測は実装しない。

技術スタック

- Flutter / Dart
- ローカル DB: sqflite
- 写真取得: image_picker
- 状態管理: setState のみ (Prototype 1)

ドキュメントの場所

内容	パス
仕様の正 (何を作るか)	`docs/product-requirements.md`
docs 配下の索引	`docs/README.md`
レイヤー分割・依存の向き	`docs/skills/clean-architecture/SKILL.md`
Widget・状態管理・コメント方針	`docs/skills/flutter/SKILL.md`
画面遷移	`docs/skills/app-flow/SKILL.md`
テスト方針	`docs/skills/test/SKILL.md`
禁止事項・確認が必要な操作	`docs/skills/guardrails/SKILL.md`
ドキュメント配置ルールの正	`README.md`

最上位ルール

- `main` / `develop` で直接作業しない。必ず `feature/<やること>` を切る
- 変更は Pull Request にする。ユーザーのレビューなしにマージしない
- 1つの関心事 = 1つの置き場所。同じ内容を2箇所に書かない
- skill の内容と実装がずれる場合は、skill を変更する前にユーザーへ確認する
- 既存ファイルを要約したり情報を削ったりしない

作業ルール

1. 画面・フォーム・一覧・ナビゲーション・スタイルを変更する前に
`docs/skills/app-flow/SKILL.md` と `docs/skills/flutter/SKILL.md` を確認する
2. 新しいクラス・ファイル・ディレクトリを作る前に
`docs/skills/clean-architecture/SKILL.md` を確認する
3. 血糖値・日時・メモ・写真の入力仕様や上限値を変更する前に
`docs/product-requirements.md` を確認する
4. 画面遷移を追加・変更する前に
`docs/skills/app-flow/SKILL.md` を確認し、Mermaid 図も更新する
5. テストを追加・変更する前に
`docs/skills/test/SKILL.md` を確認する
6. 仕様変更・機能追加・データ削除・有料サービス導入の前に
`docs/skills/guardrails/SKILL.md` を確認する
7. 新しい md を作る前に `README.md` のドキュメント配置ルール表を確認する

タスクの進め方

1. 次に実行するタスクを確認する
2. 実装内容を簡潔に説明し、ユーザーの許可を得る
3. そのタスクだけ実装する
4. テストを行う
5. 完了報告をする（下の形式）
6. ユーザーの動作確認と OK を待つ。OK が出るまで次のタスクへ進まない

完了報告の形式

- 実装したこと
- 変更したファイル一覧
- コードのポイント（初心者が理解するとよい Flutter/Dart のポイント）
- テスト結果
- ユーザーが確認すること（どこを操作して何を見ればよいか）

ポイント

- 「ドキュメントの場所」は表にしてパスで書く。AI がそのまま開けます
- 作業ルールの行為は具体語で。「UI を触る前に」ではなく「画面・フォーム・一覧・ナビゲーション・スタイルを変更する前に」
- ここにルールの中身は書かない。中身は skill 側。入口が厚いと肝心の案内が埋もれます

4. 作る SKILL 5つ

パス	中身	素材
docs/skills/ clean-architecture/ SKILL.md	lib 配下の層と責務／依存は必ず内向き／どこに何を置くか models=データの形とルール / repositories=保存と取得の窓口 / services=外部（DB・カメラ）との接点 / pages=画面 / widgets= 画面をまたぐ部品	新規に決める 現状 models/pages/services のみ。repositories が無い
docs/skills/ flutter/SKILL.md	状態管理は setState のみ（理由・見直す条件） / Widget は build を分割する / 日本語コメントの方針 / 依存追加は確認を取る	development-rules 10章 + 未記録の決定を追記
docs/skills/ app-flow/SKILL.md	画面遷移を Mermaid で1枚 / 画面を増やすときのルール / 戻る動作 の扱い	新規に決める
docs/skills/ test/SKILL.md	必ず書く対象（バリデーションの境界値・DB スキーマ・保存と読み出し） / 書かなくていいもの（見た目・パディング） / 実行コマンド	新規に決める 現状「必要なテストを行う」しか書かれていない
docs/skills/ guardrails/SKILL.md	確認が必要な操作 / 任せてよい判断 + 決定の記録先 / 実装しないもの（ログイン・AI 予測）	development-rules 8章・9章 ほぼそのまま

app-flow の Mermaid たたき台

Prototype 1 の範囲で書くようになります。これを渡して、実装に合わせて直させてください。

```
```mermaid
flowchart TD
 A[アプリ起動] --> B[記録入力画面]
 B -->|日時をタップ| B1[日付ピッカー] --> B2[時刻ピッカー] --> B
 B -->|カメラで撮影 / 写真を選択| B3[写真追加
最大3枚] --> B
 B -->|記録を保存| C[入力チェック]
 C -->|NG| B4[エラー表示] --> B
 C -->|OK| D[SQLite に保存] --> B
 B -->|履歴タブ| E[カレンダー画面]
 E -->|日付を選択| F[その日の記録一覧]
 F -->|記録がない日| F1[空状態の表示]
 F -->|戻る| E
```
```

未定なので TODO を打つところ 一覧の並び順（時刻の昇順か降順か）、記録の編集・削除の可否、入力画面と履歴画面の行き来の方法（下タブか）。**決めていないことは「TODO: 要確認」と書かせる**。空欄にしておくと AI が自分の好みで実装します。

SKILL.md 共通の書き方

- 先頭に frontmatter。description は「何を定義するか」 + 「いつ使うか」の2つを必ず入れる
- 本文は簡条書き。1項目1行。断定形（「～する」「～しない」）
- 60行以内。コードを読めば分かることは書かない。書くのは**決めたこととその理由**
- 1回の指示で1ファイルだけ作る**。5個まとめて作らせない

5. README に入れるもの

| ブロック | 中身 |
|--------------|---|
| タイトルと概要 | 楽ちん報告アプリ（仮）が何を解決するか（3行） |
| スクリーンショット | docs/images/ を作って置く。![[記録入力画面]](docs/images/record_input.png)
画像の置き場所もパスルール表に1行入れておく |
| 技術スタック | Flutter / Dart / sqflite / image_picker |
| ディレクトリ構成 | lib/ と docs/ の意味を1行ずつ |
| ドキュメント配置ルール表 | 本体。配置ルールの正はこの表1箇所と明記する |
| ブランチ運用 | 次章の内容を短く。詳細は AGENTS.md 側と重複させない |
| AI で作業する場合 | 「AGENTS.md を先に読む」の1行 |

6. GitHub 運用ルール

現状はブランチが `feature/docs-restructure` の1本だけで、それがデフォルトブランチになっています。戻る先が存在しないので保険が効いていません。まずここを直します。

```
# 1) 今の状態を main として確定
git checkout feature/docs-restructure
git checkout -b main
git push -u origin main
#   GitHub の Settings → General → Default branch を main に変更

# 2) 開発の受け皿
git checkout -b develop
git push -u origin develop

# 3) 以降は毎回これ
git checkout develop
git checkout -b feature/docs-restructure-v2
# ... 作業 ...
git push -u origin feature/docs-restructure-v2
gh pr create --base develop --title "docs: 構成を再編" --body "..."
```

| ブランチ | ルール |
|----------------|--------------------------------------|
| main | 動くものだけ。直接コミット禁止。develop からの PR でのみ更新 |
| develop | 開発の合流点。feature はここから切って、ここへ PR |
| feature/<やること> | 実作業。1タスク1ブランチ。マージ後は削除 |

PR のルール（AGENTS.md に書く文言）

```
## Pull Request のルール

- 作業前に develop から feature/<やること> を切る
- コミットは1目的1コミット。docs の移動と仕様変更を同じコミットに混ぜない
- 作業が終わったら PR を作成し、ユーザーのレビューを待つ
- ユーザーが承認するまでマージしない。self-merge しない
- PR には次を書く
  - 何を変えたか（3行以内）
  - 変更したファイル一覧
  - レビューで特に見てほしい点
  - 動作確認の手順（ユーザーが何をどう触れば確認できるか）
```

.github/pull_request_template.md

```
## 何を変えたか

## 変更したファイル

## レビューで見てほしい点

## 動作確認の手順

## 関連するドキュメント
- [ ] 仕様を変えた → `docs/product-requirements.md` を更新した
- [ ] 実装方針を変えた → 該当する `docs/skills/<name>/SKILL.md` を更新した
- [ ] 画面遷移を変えた → `docs/skills/app-flow/SKILL.md` の Mermaid を更新した
```

レビューを回すコツ PR が大きいとレビューできません。「AGENTS.md を作る」だけで1 PR、「flutter の SKILL を作る」だけで1 PR に割ってください。1 PR=1ファイル くらいの粒度が、初心者がレビューを続けられる限界です。チェックリストの最後の3行は、**docs と実装がズレる事故を PR の時点で止めるための**ものです。ここが今回いちばん効きます。

7. 実行順（1ステップ1 PR）

| PR | やること | できあがるもの |
|----|---|-----------------------|
| — | main / develop を作り、デフォルトを main に | 戻れる状態 |
| 1 | PROJECT_RULES.md → AGENTS.md に改名。L29-52 の残骸を development-rules.md へ戻す | 入口が読まれる／切れた文章が復活 |
| 2 | AGENTS.md を第3章のひな形で書き直す（パス表・作業ルール・進め方・報告形式） | AI が自分で docs を選ぶようになる |
| 3 | guardrails/SKILL.md を作る（8・9章から） | 禁止事項を毎回書かなくてよくなる |
| 4 | flutter/SKILL.md を作る（10章＋未記録の決定） | 状態管理が勝手に変わらなくなる |
| 5 | clean-architecture/SKILL.md を作る | ファイルの置き場所が固定される |
| 6 | app-flow/SKILL.md を作る（Mermaid） | 画面追加時の 判断基準ができる |
| 7 | test/SKILL.md を作る | テスト量が安定する |
| 8 | development-rules.md を削除。README を書き直す（画像・パスルール表） | 重複ゼロ／地図が完成 |

PR 2 の指示（コピペ）

```
AGENTS.md を書き直します。1ファイルだけ触ってください。

構成：
- プロジェクト概要（3〜5行）
- 技術スタック
- ドキュメントの場所（表。ファイル名ではなくリポジトリルートからのパスで書く）
- 最上位ルール（ブランチ / PR / 1関心事1置き場所 / skill と実装のズレは確認）
- 作業ルール（「〇〇する前に </パス> を確認する」の番号付きリスト。
  行為は AI が自分で判定できる具体的な言葉にすること）
- タスクの進め方
- 完了報告の形式

禁止：
- ルールの中身は書かない。中身は各 SKILL.md 側に置く
- 素材にない内容を推測で足さない。不明点は「TODO: 要確認」と書いて私に質問する
- 他のファイルは変更しない

書き終えたら feature ブランチに push し、develop 宛の PR を作成してください。
マージはしないでください。私がレビューします。
```

効き目テスト（新しいセッションで）

```
このプロジェクトで「保存した記録をカレンダーから確認できる履歴画面」を追加したいです。
実装はまだしないでください。
先に、あなたがこの作業のために読むべきドキュメントを、読む順番にパスで列挙してください。
なぜそれを読むのかも一言ずつ添えてください。
```

合格ライン：AGENTS.md → docs/product-requirements.md → docs/skills/app-flow/SKILL.md → docs/skills/clean-architecture/SKILL.md → docs/skills/test/SKILL.md を自力で挙げられたら成功。挙げがなかった skill は description か作業ルールの書き方が曖昧ということです。